

Vol Calculator

Full Technical Documentation: Mathematics & Implementation

This document is the complete technical reference for the Vol Calculator project as currently built: every mathematical formula the code evaluates, why that formula (not an alternative), and exactly where in the codebase it lives. It covers the instrument model, the American-option pricing engine, every Greek, breakevens, payoff diagrams, the seven multi-leg structures, portfolio risk views (tenor, stress ladders, strike map), the visualization layer, CSV I/O, the Streamlit application, and packaging. It reflects the code as it stands today, including every correction made during review (the Black-76 futures pricing fix, the Rho unit-scale fix, and the \$k dollar-conversion layer).

Source: `vol_calculator/` package, `app.py`, `launcher.py`

Contents

1. System Overview & Architecture
2. Instrument & Market Model
3. Pricing Engine: Black-76 and the Barone-Adesi-Whaley Approximation
4. Greeks — Mathematics, Units, and Implementation
5. Breakevens
6. Payoff Diagrams
7. Multi-Leg Structures
8. Portfolio Risk Views: Tenor, Stress Ladders, Strike Map
9. Visualization Layer
10. CSV Import / Export
11. Streamlit Application
12. Packaging & Distribution
13. Appendix: Full Function Reference

1. System Overview & Architecture

Vol Calculator is a risk engine for a futures + American-option portfolio: it prices positions, computes every Greek by finite differences on the real pricer, and builds every scenario view (ladders, surfaces, stress grids, payoff diagrams, breakevens) by re-running that same pricing pipeline under systematically shocked inputs. There is a single source of truth for all of this — the `vol_calculator` Python package — consumed identically by a Jupyter notebook, a plain script, and a Streamlit web application. No calculation logic is duplicated in any of those three surfaces.

1.1 Package layout

Module	Contents
<code>market.py</code>	Market — the live conditions a portfolio is priced against.
<code>instruments.py</code>	Portfolio, Future, AmericanOption, Structure — the position model.
<code>pricing.py</code>	AmericanOptionPricer — the QuantLib pricing call for one option leg.
<code>greeks.py</code>	GreekEngine and the <code>shift_portfolio_vols</code> / <code>shift_time</code> shock helpers.
<code>scenarios.py</code>	Ladders, surfaces, breakevens, payoff diagrams, tenor report, stress ladders, strike map.
<code>structures.py</code>	Factory functions for the seven named multi-leg structures.
<code>plotting.py</code>	Matplotlib/seaborn visualizations for every scenario function.
<code>io.py</code>	Portfolio ↔ CSV round-trip, generic report CSV export.
<code>app.py</code>	The Streamlit application (thin UI layer over the package).
<code>launcher.py</code>	Programmatic Streamlit bootstrap used to build the standalone executable.

1.2 Data flow

Every computation in the system follows the same shape:

```
Market (F, r, multiplier) + Portfolio (positions)
↓ price(market) on each position
Portfolio.value(market) → a single number
↓ bump an input, revalue, difference
GreekEngine.* → Delta, Gamma, Vega, Theta, Rho, Volga, Vanna
↓ sweep many bumped states
scenarios.* → DataFrames (ladders, surfaces, breakevens, payoff, stress, strike map)
↓
plotting.* / app.py → charts and the interactive UI
```

Because Future, AmericanOption, and Structure all expose the same `price(market)`, `shift_vol(dv)`, and `shift_time(days)` interface, every function above the instrument layer is completely agnostic to what it's holding — a portfolio of 40 mixed single legs and multi-leg structures is priced and differentiated with exactly the same code path as a single option.

2. Instrument & Market Model

2.1 Market

```
@dataclass
class Market:
    futures_price: float
    interest_rate: float
    contract_multiplier: float = 50.0  # $ per 1c/bu move per contract
```

A snapshot of live conditions: the futures price F , the risk-free rate r , and a `contract_multiplier`. The multiplier's default of 50 comes directly from the underlying's quoting convention used throughout this project: `futures_price` is quoted in cents/bu (e.g. 528.75 = \$5.2875/bu), and the standard CBOT grain contract covers 5,000 bushels, so:

$$\text{\$ per 1¢/bu move per contract} = 1¢ \times 5,000 \text{ bu} = \$50$$

This single constant is what turns the model's raw price-unit arithmetic into real dollars wherever a dollar figure is reported (Section 4.8).

2.2 Future

$$\text{Future.price}(\text{market}) = \text{quantity} \times F$$

A linear position — no optionality, no discounting. `shift_vol` and `shift_time` are no-ops (return self) since a future has no volatility or expiry to shift.

2.3 AmericanOption

A single option leg: `quantity`, `strike` (K), `expiry_days` (τ), `option_type`, `volatility` (σ). Critically, volatility is a property of the *leg*, not the market — there is no single shared implied vol. This is what lets one portfolio hold legs priced off different points on a skew (e.g. a strangle's put and call trading at different vols) without any special casing anywhere else in the code.

$$\text{AmericanOption.price}(\text{market}) = \text{quantity} \times \text{AmericanOptionPricer}(F, K, r, \sigma, \tau, \text{type}).\text{price}()$$

`shift_vol(dv)` returns a new leg with `volatility + dv`; `shift_time(days)` returns a new leg with `expiry_days = max(expiry_days - days, 1)` — time can only move forward, and is floored one day short of literal expiry to avoid asking the pricer to value a 0-day American option (Section 6 explains the consequence of this floor and how payoff diagrams route around it).

2.4 Structure — the composite pattern

A `Structure` (call spread, straddle, butterfly, ...) is *not* a new kind of priceable primitive — it is a labeled bag of `AmericanOption/Future` legs that implements the exact same three-method interface:

$$\begin{aligned} \text{Structure.price}(\text{market}) &= \sum \text{leg.price}(\text{market}) \text{ for leg in legs} \\ \text{Structure.shift_vol}(\text{dv}) &= \text{Structure}([\text{leg.shift_vol}(\text{dv}) \text{ for leg in legs}], \text{name}) \\ \text{Structure.shift_time}(\text{days}) &= \text{Structure}([\text{leg.shift_time}(\text{days}) \text{ for leg in legs}], \text{name}) \end{aligned}$$

Because it quacks exactly like any other instrument, a `Structure` can be added directly to a `Portfolio` alongside bare legs, and every single function built on top — every Greek, every ladder, every plot — works on it with **zero** special casing. This is the mechanism that lets the seven structure factories (Section 7) be pure

convenience wrappers with no new pricing math of their own.

2.5 Portfolio

```
Portfolio.value(market) =  $\sum$  position.price(market) for position in positions
```

An ordered list of positions (bare legs and/or Structures). This one polymorphic sum is the foundation everything else in the system is built on.

3. Pricing Engine: Black-76 and the Barone-Adesi-Whaley Approximation

3.1 Why the underlying process is Black-76, not Black-Scholes-Merton

The instrument being priced is a *futures* option, not a *spot* option, and that distinction changes the correct pricing model. A futures price has zero drift under the risk-neutral measure (entering a futures contract costs nothing, so its expected discounted payoff must be zero) — equivalently, a futures price behaves exactly like a spot asset paying a continuous dividend yield $q = r$. Fischer Black's 1976 model prices European options on such an underlying:

$$c = e^{-rT} [F \Phi(d_1) - KN(d_2)] \quad p = e^{-rT} [KN(-d_2) - F \Phi(-d_1)]$$
$$d_1 = [\ln(F/K) + \sigma^2 T/2] / (\sigma \sqrt{T}) \quad d_2 = d_1 - \sigma \sqrt{T}$$

The critical feature: there is **no r inside d_1** . F is already the risk-neutral expectation of the future spot price, so it doesn't get drifted forward before discounting — the only place r appears is the single e^{-rT} discount factor on the whole expected payoff. This was a real, verified bug in an earlier version of this code: the pricer originally fed the futures price into QuantLib's plain `BlackScholesProcess` (which implicitly assumes $q = 0$, drifting F upward at the full rate r like a non-dividend stock), which produced measurably wrong prices — on the demo book, the total portfolio value shifted from -500.24 to -502.38 after the fix, with the near-the-money leg alone moving by about 9% of its own value.

The fix, as implemented

```
process = ql.BlackProcess(
    ql.QuoteHandle(ql.SimpleQuote(self.F)),
    ql.YieldTermStructureHandle(risk_free),
    ql.BlackVolTermStructureHandle(vol_curve)
)
```

`ql.BlackProcess` is QuantLib's purpose-built process for exactly this case — no separate dividend curve to keep in sync, $q = r$ is built in. This is what `pricing.py` uses today.

3.2 American exercise: the Barone-Adesi-Whaley approximation

Because the option can be exercised at any time up to expiry (not just at expiry), its value exceeds the European price above by an *early-exercise premium*. Barone-Adesi & Whaley (1987) approximate this premium by finding a critical underlying price S^* above which (for a call) immediate exercise is optimal, and fitting a quadratic correction term calibrated so the American and European values, and their derivatives, match smoothly at S^* . The full American value is then:

$$\text{American value} = \text{European (Black-76) value} + \text{early-exercise premium}(S^*)$$

This is a closed-form *approximation*, not a lattice or PDE solve — it trades a small amount of accuracy (most pronounced very close to expiry, at the money) for being fast enough to call thousands of times per Greek/scenario computation without the book becoming slow to use interactively. QuantLib implements this as `BaroneAdesiWhaleyApproximationEngine`, which is what `AmericanOptionPricer` attaches to the process above.

3.3 Full pricer call, as implemented

```
calc_date = ql.Date.todaysDate()
ql.Settings.instance().evaluationDate = calc_date
expiry = calc_date + self.days

payoff = ql.PlainVanillaPayoff(Call/Put, self.K)
exercise = ql.AmericanExercise(calc_date, expiry)
option = ql.VanillaOption(payoff, exercise)

risk_free = ql.FlatForward(calc_date, self.r, ql.Actual365Fixed())
vol_curve = ql.BlackConstantVol(calc_date, ql.NullCalendar(), self.vol, ql.Actual365Fixed())
process = ql.BlackProcess(SimpleQuote(self.F), risk_free, vol_curve)
engine = ql.BaroneAdesiWhaleyApproximationEngine(process)

option.setPricingEngine(engine)
return option.NPV()
```

Expiry is a day-count offset from *today*, not a fixed calendar date — this is why time decay (Theta, and the time axis of every scenario view) is simulated by decrementing `expiry_days` rather than moving the evaluation date. The vol and rate curves are both flat (no term structure, no smile) — each leg's own σ stands in for its point on a skew.

4. Greeks — Mathematics, Units, and Implementation

QuantLib does not expose an analytic Greek for the Barone-Adesi-Whaley engine, so every sensitivity here is computed the same way: bump an input, reprice the whole portfolio (or a shifted copy of it), and take a finite difference. Let $V(F, \{\sigma_i\}, r)$ denote total portfolio value (`Portfolio.value(market)`). Bump constants: `SPOT_BUMP`=0.001 (0.1% of F), `VOL_POINT`=0.01 (1 vol point), `PERCENT`=0.01 (1% rate move).

4.1 Delta — contracts

```
ds = F × SPOT_BUMP
Delta = [V(F+ds) - V(F-ds)] / (2ds) ≈ ∂V/∂F
```

A central-difference estimate of $\partial V/\partial F$. Since `Future.price` = $q \cdot F$ contributes exactly q , and each option's $\partial \text{price}/\partial F$ is its own dimensionless delta, this equals **Delta** = $q_{\text{future}} + \sum q_i \delta_i$ — a futures-equivalent **contract count**, with no unit conversion needed.

4.2 Gamma — Δ delta-contracts per 1¢/bu

```
Gamma = [V(F+ds) - 2V(F) + V(F-ds)] / ds2 ≈ ∂2V/∂F2 = ∂Delta/∂F
```

The standard 3-point second-derivative stencil, taken directly against F (quoted in cents/bu) — already exactly "change in delta-contracts for a 1¢/bu move," no rescale required.

4.3 Vega — \$ per 1 vol point

```
dv = VOL_POINT (every leg's σ shifted ±dv, in parallel)
raw = [V(σ+dv) - V(σ-dv)] / (2dv) ≈ ∂V/∂σ
Vega = raw × VOL_POINT
```

The raw derivative is per unit (100-point) decimal vol; multiplying by `VOL_POINT` rescales it to **\$ change in V for a 1 percentage-point parallel shift** in every leg's vol.

4.4 Theta — \$ for one day of decay

```
Theta = V(portfolio, 1 day closer to expiry) - V(portfolio, today)
```

Not a continuous derivative — a literal one-step simulation via `shift_time(portfolio, 1)`, $F/\sigma/r$ held fixed. Sign convention: a long option (losing time value) shows up negative.

4.5 Rho — \$ per 1% rate move (corrected)

```
dr = BASIS_POINT = 0.0001
raw = [V(r+dr) - V(r-dr)] / (2dr) ≈ ∂V/∂r
Rho = raw × PERCENT (PERCENT = 0.01)
```

The finite-difference *step* stays a tiny 1bp for numerical accuracy, but the *rescale* at the end was corrected from `BASIS_POINT` (giving \$/1bp) to `PERCENT` (giving \$/1%, i.e. 100× larger) — the desk convention requires the latter. This was the one Greek among the five where the original code's convention genuinely didn't match spec, versus Delta/Gamma/Vega/Theta which already matched by construction.

4.6 Volga & Vanna — second-order / cross Greeks

Greek	Formula	Meaning
Volga	$[V(\sigma+dv) - 2V(\sigma) + V(\sigma-dv)] / dv^2 \times \text{VOL_POINT}$	$\partial^2 V / \partial \sigma^2$ — convexity of value to vol.
Vanna-V	$[\text{Vega}(F+ds) - \text{Vega}(F-ds)] / (2ds)$	$\partial^2 V / \partial F \partial \sigma$, computed as a spot-difference of Vega.
Vanna-D	$[\text{Delta}(\sigma+dv) - \text{Delta}(\sigma-dv)] / (2dv)$	The same cross-partial, computed as a vol-difference of Delta instead — by symmetry of mixed partials, Vanna-D and Vanna-V should closely agree; small gaps are pure finite-difference error and serve as a built-in consistency check.

4.7 Two reporting conventions: report() vs. report_dollars()

`GreekEngine.report()` returns every Greek exactly as derived above — Delta/Gamma in contracts, Vega/Theta/Rho in raw price-unit dollars. `GreekEngine.report_dollars()` is the desk-facing view: it leaves Delta and Gamma untouched (they are already correct as contract counts — multiplying them by the contract multiplier would make them wrong, not right) and rescales only Value/Vega/Theta/Rho:

```
to_k = market.contract_multiplier / 1000
Value_$k = portfolio.value(market) * to_k
Vega_$k = Vega * to_k
Theta_$k = Theta * to_k
Rho_$k = Rho * to_k
```

With the default multiplier of 50 (Section 2.1), this is exactly "\$ per contract, in thousands." Volga and the Vanna terms are left in their raw form in both reports since they weren't part of the desk's five-metric specification.

5. Breakevens

5.1 Quadratic (Taylor) breakeven — breakevens_quadratic

A second-order Taylor expansion of P&L around current spot:

$$\Delta V \approx \text{Delta} \cdot \Delta F + \frac{1}{2} \cdot \text{Gamma} \cdot \Delta F^2 + \text{Theta}$$

Setting $\Delta V = 0$ gives a quadratic in ΔF , solved directly:

$$\Delta F = [-\text{Delta} \pm \sqrt{\text{Delta}^2 - 2 \cdot \text{Gamma} \cdot \text{Theta}}] / \text{Gamma}$$

Breakeven price levels are $F \pm \Delta F$. Cheap (reuses already-computed Delta/Gamma/Theta), but degrades for large moves or kinked payoffs (butterflies) since it's a local, smooth approximation.

5.2 Numerical breakeven — breakevens_numerical

Finds the spot move where next-day P&L, computed with the *real pricer*, exactly crosses zero:

$$f(\Delta F) = V(\text{shift_time}(\text{portfolio}, 1), F + \Delta F) - V(\text{portfolio}, F)$$

Implementation: scan a grid of 200 points across $\pm 50\%$ of F , find sign changes in f , then refine each with `scipy.optimize.brentq` bisection. No Taylor error, and the multi-point scan (rather than assuming one crossing) correctly reports `None` for a side that genuinely has no breakeven — verified on the demo book, where positive Theta means the book is already profitable from one day of decay with no downside move needed at all, and the numerical method correctly returns no downside root while the quadratic method reports a spurious one right next to current spot.

6. Payoff Diagrams

6.1 Pure intrinsic value at expiry — `payoff_at_expiry`

```
Call:  $\max(F - K, 0)$  Put:  $\max(K - F, 0)$  Future:  $q \cdot F$ 
```

Computed **directly in Python**, leg by leg, rather than by asking the QuantLib pricer to value a 0-day American option (an edge case worth avoiding entirely, since `AmericanOption.shift_time` floors at 1 day and can never literally reach it). P&L is intrinsic value minus today's value — the same convention used throughout the ladders, and one that correctly nets out a Future leg's entry price with no extra bookkeeping.

6.2 Time-stepped payoff diagram — `payoff_diagram`

For each requested day-offset (e.g. [0, 7, 14, 30]), `shift_time` moves the whole book that many days forward and reprices with the *real* pricer — so, unlike `payoff_at_expiry`, time value is preserved for legs that haven't reached expiry. This means the final time-step column and the pure-intrinsic curve will *not* coincide exactly (the former is capped one day short of expiry by the same floor noted above, and legs at different tenors within one book will differ even more) — call `payoff_at_expiry` separately for the true final curve.

7. Multi-Leg Structures

Every structure factory is pure construction convenience: it assembles correctly-signed `AmericanOption` legs and wraps them in a `Structure` — none of them add new pricing math. Quantity sign convention: positive quantity is the conventional "long" version of the structure for every factory except `collar`, which takes independent `put_qty/call_qty` so it can size the two legs differently against an existing position.

Structure	Legs	Construction rule
<code>call_spread</code>	2	Long $qty \times K_{long}$, short $qty \times K_{short}$ ($K_{long} < K_{short}$), both calls, same expiry.
<code>put_spread</code>	2	Long $qty \times K_{long}$, short $qty \times K_{short}$ ($K_{long} > K_{short}$), both puts, same expiry.
<code>straddle</code>	2	$qty \times \text{call} + qty \times \text{put}$, same strike & expiry, independent per-leg vols.
<code>strangle</code>	2	$qty \times \text{put at } K_{put} + qty \times \text{call at } K_{call}$ ($K_{put} < K_{call}$).
<code>butterfly</code>	3	$qty \times K_1, -2qty \times K_2, qty \times K_3$ ($K_1 < K_2 < K_3$), same type; ratio (1,2,1) overridable.
<code>collar</code>	2	$put_qty \times K_{put} + call_qty \times K_{call}$ ($K_{put} < K_{call}$), independently signed/sized.
<code>calendar_spread</code>	2	$-qty \times \text{near expiry} + qty \times \text{far expiry}$, same strike & type ($near_days < far_days$) — already fully supported by <code>AmericanOption</code> 's independent per-leg <code>expiry_days</code> , no new mechanism needed.
<code>combo</code>	n	Generic wrapper: <code>Structure(legs, name)</code> for any custom/3-way combination that doesn't fit a named template — deliberately unopinionated since 3-way conventions vary by desk.

Every factory validates its strike ordering (e.g. `call_spread` raises `ValueError` if $K_{long} \geq K_{short}$) before constructing legs.

8. Portfolio Risk Views: Tenor, Stress Ladders, Strike Map

8.1 Tenor-bucketed Greeks — `report_by_tenor`

Every Structure in the portfolio is flattened to its individual legs first (so a calendar spread's two legs land in two different buckets, not one), then grouped by `expiry_days` (a separate "Futures" bucket holds legs with no expiry). `GreekEngine.report()` runs once per bucket plus once for the whole book (the "Total" row) — no new Greek math, purely a partition applied before calling existing code.

8.2 Stress ladders — `stress_table / stress_report`

Generalizes the spot/vol ladders into "stress one variable, hold others fixed, read off one Greek":

```
stress_table(portfolio, market, greek, axis, magnitudes)
```

Axis	Shock	Sign
price	market.futures_price + level (absolute cents)	± each magnitude
time	shift_time(portfolio, level)	Forward only (magnitudes as-is)
vol	shift_portfolio_vols(portfolio, level/100)	± each magnitude (vol points)

`stress_report()` runs the full required grid in one call — Delta × {price, time, vol}, Gamma × {price, time, vol}, Vega × {price, time}, Theta × {price, time} — 10 combinations, each a DataFrame keyed "<greek>_<axis>".

8.3 Strike map — `strike_map`

A pivot of every option leg (Structures flattened, Future legs excluded — they have no strike):

```
pivot_table(index='Strike', columns='Expiry', values=quantity_or_delta,
aggfunc='sum', fill_value=0)
```

`weight="quantity"` (default) sums raw position size per strike/expiry cell; `weight="delta"` instead computes each leg's own Delta in isolation (wrapped in a fresh single-leg Portfolio) before summing — showing net directional exposure by strike rather than raw contract count.

9. Visualization Layer

Function	Chart	Source data
plot_surface	Heatmap, spot move % × vol shift (pts), diverging colormap centered at 0	spot_vol_surface
plot_strike_map	Heatmap, strike × expiry	strike_map
plot_stress	Single line chart, one Greek vs. one stress axis	stress_table
plot_stress_report	5×2 small-multiples grid, all 10 required stress combinations	stress_report
plot_payoff	Overlaid line chart: one line per time step + a dashed "At Expiry" curve, vertical reference line at current spot	payoff_diagram + payoff_at_expiry

All five call `plt.show()` internally; in the Streamlit app this is a harmless no-op under the Agg backend, and the same figure is immediately retrieved via `plt.gcf()` and rendered with `st.pyplot()` (Section 11) — the plotting functions themselves needed no changes to work in both contexts.

10. CSV Import / Export

10.1 Portfolio schema — one row per leg

Column	Meaning
structure_id	Empty for a bare leg; shared integer across all legs of one Structure.
structure_type	The Structure's .name (e.g. "Straddle(qty=5, K=530, days=64)"); empty for bare legs.
instrument_type	"Future" or "AmericanOption".
quantity, strike, expiry_days, option_type, volatility	Blank strike/expiry/type/vol for Future rows.

`portfolio_from_csv` regroups rows sharing a `structure_id` back into one `Structure`, adding it to the portfolio at the position of its *first* row — this preserves the exact original leg ordering, including any interleaving of bare legs and structures. Verified by round-trip: exporting and reimporting a mixed portfolio (bare legs interleaved with a calendar spread) reproduced identical position order, identical portfolio value, and an exact match on every entry in `GreekEngine.report()`.

10.2 Report export — `export_report`

Every scenario function already returns a pandas `DataFrame`, so export is nearly free: `export_report(df, path)` creates the destination folder if needed and calls `df.to_csv(path)`.

11. Streamlit Application

app.py is a thin UI layer: it holds no pricing or Greek logic of its own, only widget wiring around the `vol_calculator` package's public functions. Portfolio and market state live in `st.session_state` so they persist across Streamlit's rerun-on-every-interaction model; the app opens pre-loaded with the same 3-leg demo portfolio used throughout this document.

11.1 Sidebar

- **Market** — live futures price / rate / contract multiplier inputs, instantly repricing everything.
- **Add Leg** — a dropdown across all 9 buildable types (Future, American Option, and the 7 named structures), each rendering its own input form via a declarative `LEG_BUILDERS` spec that maps directly onto each factory's parameter list from Section 7.
- **Current Positions** — every leg/structure with a per-row remove button.
- **Portfolio File** — CSV upload (writes to a temp file, calls `portfolio_from_csv`) and download (calls `portfolio_to_csv`), wired directly to Section 10.

11.2 Main area — 8 tabs

Tab	Content
Portfolio	Position list and total value.
Greeks	<code>report()</code> and <code>report_dollars()</code> side by side, plus <code>report_by_tenor()</code> .
Ladders	<code>spot_ladder</code> and <code>vol_ladder</code> , adjustable range controls, CSV download.
Surface	<code>plot_surface</code> with a selectable metric.
Stress	<code>plot_stress_report</code> — the full 10-panel grid.
Strike Map	<code>plot_strike_map</code> , quantity/delta toggle.
Payoff	<code>plot_payoff</code> , adjustable time-step list.
Breakevens	<code>breakevens_quadratic</code> and <code>breakevens_numerical</code> side by side.

Verified with Streamlit's AppTest harness: all 8 tabs execute without exception on load, form-submitted leg additions correctly grow the portfolio (a Straddle add produced a Structure instance as expected), and remove buttons correctly shrink it.

12. Packaging & Distribution

12.1 Standalone executable (built)

`launcher.py` boots Streamlit programmatically (`streamlit.web.cli.main()` with `sys.argv` set, rather than shelling out to a `streamlit` command that wouldn't exist inside a frozen executable) and opens the default browser on a short timer. `PyInstaller` bundles this, `app.py` (as a required data file — Streamlit re-executes its entry script from disk on every rerun, so it can't be frozen to bytecode the normal way), and the entire `vol_calculator` package plus all dependencies (QuantLib, pandas, matplotlib, seaborn, scipy, Streamlit's full static asset tree) into a single distributable folder.

Verified end-to-end with a real headless-browser session against the built `.exe` (not just an HTTP health check): the demo portfolio loaded, portfolio value rendered as -502.38 (matching the corrected Black-76 calculation exactly), and the Surface tab's seaborn heatmap rendered correctly — confirming QuantLib's compiled extension survived the freeze.

Caveat: only `vol_calculator`'s compiled bytecode is meaningfully hidden inside the executable. `app.py` is extracted as a plain-text file to a temp folder at runtime (a consequence of how Streamlit executes scripts), so it is not visible in the distributed folder but is technically recoverable from `%TEMP%` while the app is running.

12.2 Hosted deployment (planned, not yet built)

A private GitHub repository deployed to Streamlit Community Cloud, with an in-app password gate backed by Streamlit's Secrets feature as a platform-independent access control — discussed and agreed as the next step, not implemented as of this document.

13. Appendix: Full Function Reference

Module.Function	Signature	Returns
market.Market	(futures_price, interest_rate, contract_multiplier=50.0)	dataclass instance
instruments.Portfolio	()	positions: list
instruments.Future	(quantity)	priceable instrument
instruments.AmericanOption	(quantity, strike, expiry_days, option_type, volatility)	priceable instrument
instruments.Structure	(legs, name=None)	composite priceable instrument
pricing.AmericanOptionPricer	(futures_price, strike, rate, volatility, expiry_days, option_type)	.price() -> float
greeks.GreekEngine.delta/gamma/vega/theta/rho/volga/vanna_delta/vanna_vega	(portfolio, market)	float
greeks.GreekEngine.report / summary	(portfolio, market)	dict / printed
greeks.GreekEngine.report_dollars / summary_dollars	(portfolio, market)	dict / printed (\$k view)
greeks.shift_portfolio_vols / shift_time	(portfolio, dv days)	new Portfolio
scenarios.spot_ladder / vol_ladder	(portfolio, market, low, high, step)	DataFrame
scenarios.spot_vol_surface	(portfolio, market, metric, ...)	DataFrame (grid)
scenarios.breakevens_quadratic / breakevens_numerical	(portfolio, market, ...)	{'downside','upside'}
scenarios.payoff_at_expiry / payoff_diagram	(portfolio, market, ...)	DataFrame
scenarios.report_by_tenor	(portfolio, market)	DataFrame
scenarios.stress_table / stress_report	(portfolio, market, ...)	DataFrame / dict of DataFrame
scenarios.strike_map	(portfolio, market=None, weight='quantity')	DataFrame (pivot)
structures.call_spread/put_spread/traddle/strangle/butterfly/collar/calendar_spread/combo	see Section 7	Structure
plotting.plot_surface/plot_strike_map/plot_stress/plot_stress_report/plot_payoff	see Section 9	renders a chart
io.portfolio_to_csv / portfolio_from_csv / export_report	see Section 10	None / Portfolio / None

End of document. This reflects the codebase as built through the Streamlit application and standalone packaging phases — every formula above has a corresponding, currently-running implementation cited next to it.